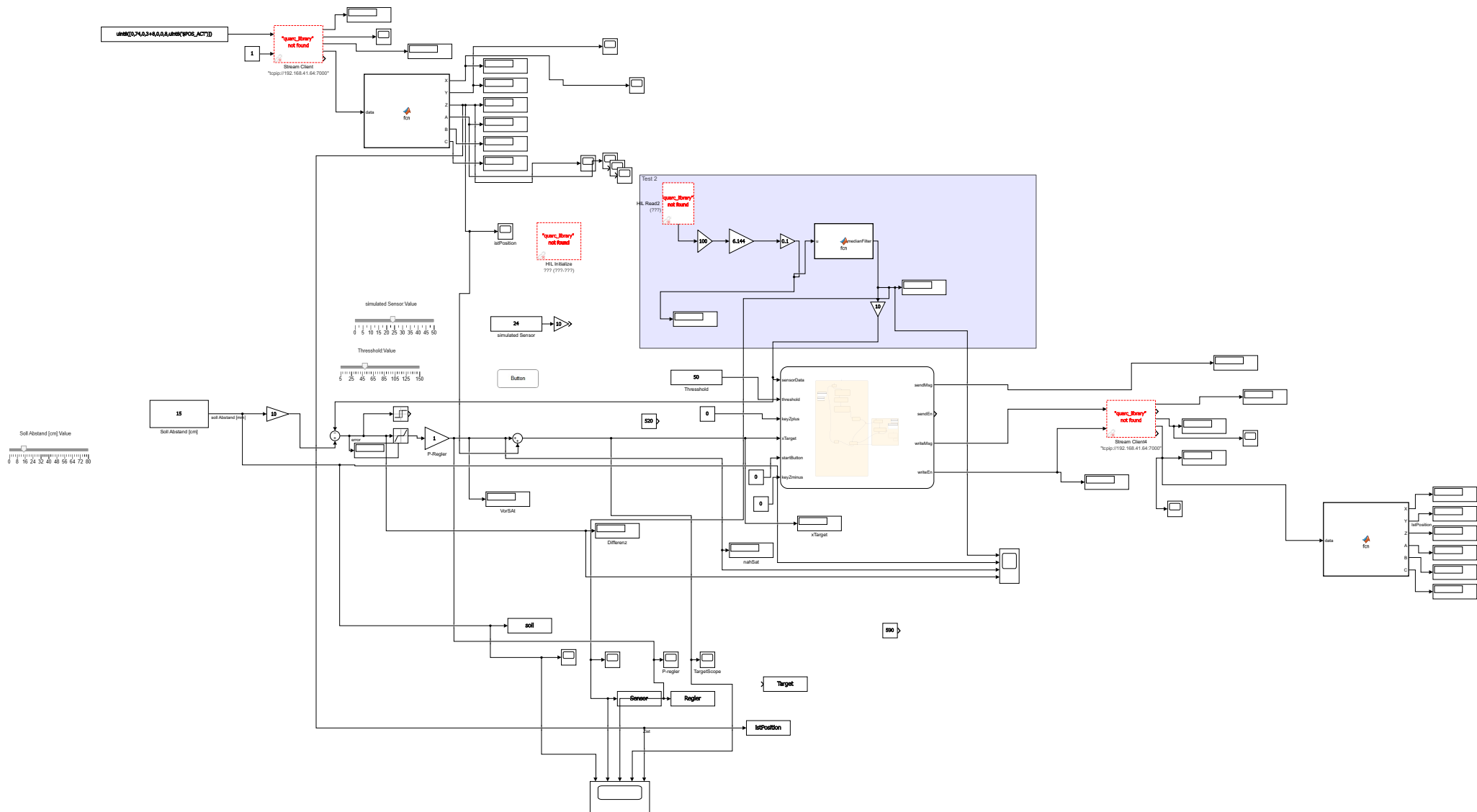
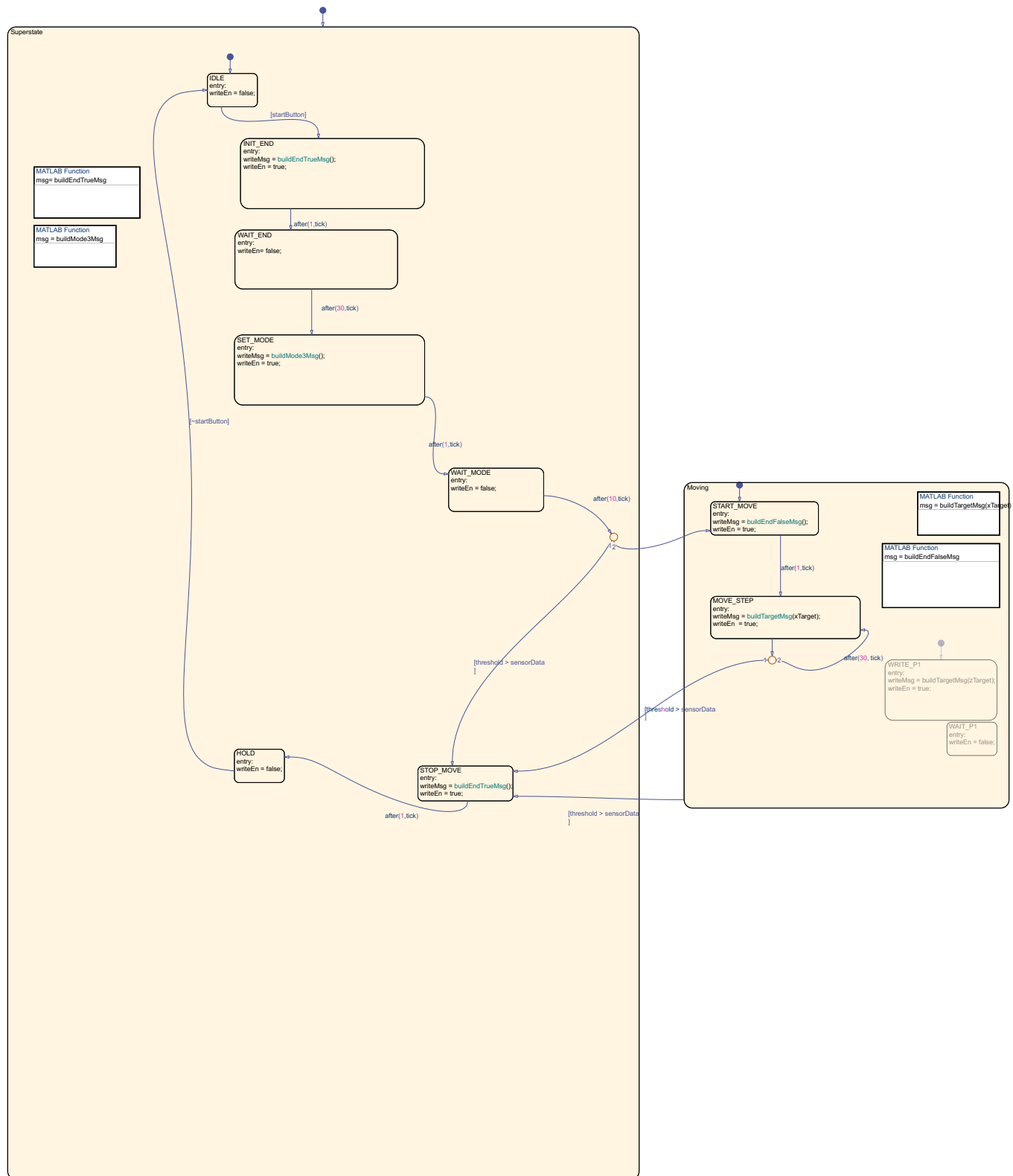






```

function [X,Y,Z,A,B,C] = fcn(data)

% Standardwerte
X = NaN;
Y = NaN;
Z = NaN;
A = NaN;
B = NaN;
C = NaN;

% Empfangsdaten in einen Vektor umwandeln
data = data(:);

% Positionen suchen
ix = find(data == 88, 1, 'first'); % X
iy = find(data == 89, 1, 'first'); % Y
iz = find(data == 90, 1, 'first'); % Z
ia = find(data == 65, 1, 'first'); % A
ib = find(data == 66, 1, 'first'); % B
ic = find(data == 67, 1, 'first'); % C

% Zahlen auslesen
if ~isempty(ix)
    X = parseNumber(data, ix + 1);
end

if ~isempty(iy)
    Y = parseNumber(data, iy + 1);
end

if ~isempty(iz)
    Z = parseNumber(data, iz + 1);
end

if ~isempty(ia)
    A = parseNumber(data, ia + 1);
end

if ~isempty(ib)
    B = parseNumber(data, ib + 1);
end

if ~isempty(ic)
    C = parseNumber(data, ic + 1);
end

end

function value = parseNumber(data, startIndex)

value = 0;

n = numel(data);
idx = startIndex;

% Sicherstellen, dass idx ein Skalar ist
idx = idx(1);

```

```

% Leerzeichen überspringen
while idx <= n
    if data(idx) ~= 32
        break;
    end
    idx = idx + 1;
end

% Vorzeichen
sgn = 1;

if idx <= n
    if data(idx) == 45      % '-'
        sgn = -1;
        idx = idx + 1;
    elseif data(idx) == 43 % '+'
        idx = idx + 1;
    end
end

% Ganzzahlteil
while idx <= n
    if data(idx) < 48 || data(idx) > 57
        break;
    end

    value = value * 10 + double(data(idx) - 48);
    idx = idx + 1;
end

% Nachkommastellen
if idx <= n && data(idx) == 46      % '.'
    idx = idx + 1;

    factor = 0.1;

    while idx <= n
        if data(idx) < 48 || data(idx) > 57
            break;
        end

        value = value + double(data(idx) - 48) * factor;
        factor = factor * 0.1;
        idx = idx + 1;
    end
end

value = sgn * value;

end

```

```

function [X,Y,Z,A,B,C] = fcn(data)

% Standardwerte
X = NaN;
Y = NaN;
Z = NaN;
A = NaN;
B = NaN;
C = NaN;

% Empfangsdaten in einen Vektor umwandeln
data = data(:);

% Positionen suchen
ix = find(data == 88, 1, 'first'); % X
iy = find(data == 89, 1, 'first'); % Y
iz = find(data == 90, 1, 'first'); % Z
ia = find(data == 65, 1, 'first'); % A
ib = find(data == 66, 1, 'first'); % B
ic = find(data == 67, 1, 'first'); % C

% Zahlen auslesen
if ~isempty(ix)
    X = parseNumber(data, ix + 1);
end

if ~isempty(iy)
    Y = parseNumber(data, iy + 1);
end

if ~isempty(iz)
    Z = parseNumber(data, iz + 1);
end

if ~isempty(ia)
    A = parseNumber(data, ia + 1);
end

if ~isempty(ib)
    B = parseNumber(data, ib + 1);
end

if ~isempty(ic)
    C = parseNumber(data, ic + 1);
end

end

function value = parseNumber(data, startIndex)

value = 0;

n = numel(data);
idx = startIndex;

% Sicherstellen, dass idx ein Skalar ist
idx = idx(1);

```

```

% Leerzeichen überspringen
while idx <= n
    if data(idx) ~= 32
        break;
    end
    idx = idx + 1;
end

% Vorzeichen
sgn = 1;

if idx <= n
    if data(idx) == 45      % '-'
        sgn = -1;
        idx = idx + 1;
    elseif data(idx) == 43 % '+'
        idx = idx + 1;
    end
end

% Ganzzahlteil
while idx <= n
    if data(idx) < 48 || data(idx) > 57
        break;
    end

    value = value * 10 + double(data(idx) - 48);
    idx = idx + 1;
end

% Nachkommastellen
if idx <= n && data(idx) == 46      % '.'
    idx = idx + 1;

    factor = 0.1;

    while idx <= n
        if data(idx) < 48 || data(idx) > 57
            break;
        end

        value = value + double(data(idx) - 48) * factor;
        factor = factor * 0.1;
        idx = idx + 1;
    end
end

value = sgn * value;

end

```

```

function medianFilter = fcn(u)
    persistent iWrite  buffer counter ;

    window_length = 10
    u = fix(u)

    if isempty(buffer)
        buffer = zeros(window_length,1)
        iWrite = 1
        counter = 0
    end

    buffer(iWrite) = u
    iWrite = iWrite +1
    counter = counter + 1

    if iWrite > window_length
        iWrite = 1
    end

    if counter > window_length
        counter = window_length;
    end


    medianFilter = fix(median(buffer(1:counter)));

end

```